

CowControl — Grabación de conteos con cámara

Para Keila — ajustes necesarios en tu script antes de probarlo contra el proyecto real

1. Lo que ya armaste está bien pensado

Miramos `grabador_conteo.py` y `configuracion_camara_conteo.pdf` a fondo. La idea de base es sólida: bucket privado, carpeta por campo, signed URL con vencimiento, RLS filtrando por campo, y sobre todo calcular el segundo de cada animal **contando fotogramas en vez de restar fechas** — así una pausa del conteo no desincroniza nada. Eso está bien resuelto y no hace falta tocarlo.

También vimos los videos de la netbook "Juana Manso" corriendo el script: arranca, detecta el conteo en curso, graba, y corta al finalizar — el programa en sí funciona. Lo que encontramos son **4 desajustes entre lo que el script espera y los nombres reales que ya usa el proyecto** (la app y la base de datos ya están en producción con otros nombres) — nada grave, pero si no se corrigen el script no va a poder ni leer ni escribir en las tablas correctas.

2. Los 4 ajustes necesarios

2.1 — El nombre de la tabla de lecturas

El script consulta `datos_lecturas` (con "s" al final). La tabla real en Supabase se llama `datos_lectura`, sin "s". Con el nombre actual, cualquier consulta a esa tabla devuelve error 404 (tabla no existe).

2.2 — El nombre de la columna del tag

El script usa `tag_uid` tanto en `datos_lecturas` como en `datos_animales`. En las dos tablas reales, esa columna se llama `rfid_uid`.

2.3 — `datos_lectura` no tiene columna `conteo_id` (el ajuste más importante)

Esto es distinto a un simple error de tipeo. El script filtra lecturas nuevas con `datos_lecturas?conteo_id=eq.X`, asumiendo que cada lectura sabe a qué conteo pertenece. En la base real, `datos_lectura` **no tiene ninguna columna que la relacione con un conteo** — la tabla conteos es completamente aparte. Se había diseñado agregar esa relación en su momento (una columna `sesion_id`) pero esa migración nunca se terminó de aplicar, porque el proyecto encontró otro camino para el ciclo de vida del conteo. Agregar esa columna ahora tocaría el código de la app (`conteo_screen.dart`) y es justo el tipo de cambio de arquitectura que hay que confirmar con Santiago antes, no algo para decidir solas/os.

La buena noticia es que **no hace falta ese cambio** para que tu script funcione: la tabla conteos ya tiene `campo_id` y `fecha` (el momento en que arrancó ese conteo puntual). Podés conseguir exactamente las mismas lecturas filtrando por campo + una ventana de tiempo:

```

# En vez de filtrar por conteo_id (no existe), se filtra por
# campo_id + fecha_hora >= el momento en que arranco ESE conteo.
def obtener_lecturas_nuevas(campo_id, fecha_inicio_conteo, ultimo_id):
    url = f"{SUPABASE_URL}/rest/v1/datos_lectura"
    params = {
        "campo_id": f"eq.{campo_id}",
        "fecha_hora": f"gte.{fecha_inicio_conteo}",
        "id": f"gt.{ultimo_id}",
        "select": "id,rfid_uid,fecha_hora",
        "order": "id.asc",
    }
    r = requests.get(url, headers=HEADERS_BASE, params=params, timeout=10)
    r.raise_for_status()
    return r.json()

# fecha_inicio_conteo sale de conteos.fecha, se trae una sola vez
# al arrancar la grabacion (junto con campo_id, en la misma consulta):
def obtener_campo_y_fecha(conteo_id):
    url = f"{SUPABASE_URL}/rest/v1/conteos"
    params = {"id": f"eq.{conteo_id}", "select": "campo_id,fecha"}
    r = requests.get(url, headers=HEADERS_BASE, params=params, timeout=10)
    r.raise_for_status()
    fila = r.json()[0]
    return fila["campo_id"], fila["fecha"]

```

2.4 — conteos.id es UUID, no un número (bigint)

En tus tablas nuevas (videos_conteo, video_marcas) declaraste `conteo_id bigint references conteos(id)`. Confirmamos en el Table Editor de Supabase que `conteos.id` es en realidad `uuid` (se ve como texto largo con guiones, ej.

2cadb242-b04d-4a2c-9d83-87aa8e469f90, no un número). Un `bigint` no puede referenciar una columna `uuid` — el `CREATE TABLE` directamente va a fallar. SQL corregido más abajo, sección 3.

2.5 — datos_animales no tiene columna "nombre" todavía

El script arma el texto que se "quema" en el video con `animales.get(uid, uid).nombre`. En el schema actual de CowControl, `datos_animales` fue reducido a propósito y **no incluye un campo "nombre"** (v1: solo `rfid_uid`, categoría/sexo, condición corporal, estado vital). Esto es una decisión de alcance ya tomada con Santiago, así que antes de agregar la columna hay que confirmárselo — no es una decisión técnica libre para el equipo de hardware/cámara.

Mientras eso se confirma, para que puedas seguir probando ya mismo, el script puede mostrar el `rfid_uid` (siempre existe) o la categoría (sexo) en vez de un nombre propio:

```

def cargar_animales():
    url = f"{SUPABASE_URL}/rest/v1/datos_animales"
    params = {"select": "rfid_uid,sexo"}
    r = requests.get(url, headers=HEADERS_BASE, params=params, timeout=15)
    r.raise_for_status()
    # Texto a mostrar: categoria (si esta cargada) + rfid_uid, ej. "vaca E053B619"
    resultado = {}
    for fila in r.json():
        etiqueta = fila["rfid_uid"]
        if fila.get("sexo"):
            etiqueta = f'{fila["sexo"]} {etiqueta}'
        resultado[fila["rfid_uid"]] = etiqueta
    return resultado

```

3. SQL corregido de las tablas nuevas

Mismo diseño que ya tenías (nada cambia en la lógica), solo con conteo_id como uuid:

```
create table videos_conteo (
  id bigint generated always as identity primary key,
  conteo_id uuid not null references conteos(id),
  path text not null,
  estado text default 'listo',
  created_at timestamptz default now()
);

create table video_marcas (
  id bigint generated always as identity primary key,
  video_id bigint not null references videos_conteo(id),
  lectura_id bigint not null references datos_lectura(id),
  conteo_id uuid not null references conteos(id),
  segundo float8 not null
);
```

Importante: no corras estas dos tablas contra el proyecto real de Supabase todavía — coordínalo primero con Santiago (una sola base compartida, y crear tablas ahí es una decisión que se toma en conjunto, no algo para aplicar en soledad). Sí podés dejar el SQL listo y probar toda la parte de grabación local (sin subir nada) mientras tanto.

4. Política de RLS del bucket — corregida

Tu documento dejaba esto como ejemplo, aclarando que había que reemplazar `perfiles` por la tabla real que vincula usuario con campo. Esa tabla ya existe: es `datos_campo` — cada fila tiene `productor_id` (el uuid del usuario logueado) y `id` (el campo). No existe ninguna tabla "perfiles" en el proyecto.

```
-- Que cada usuario solo pueda generar signed URL de los videos de SU campo:
create policy "usuarios ven videos de su campo"
on storage.objects
for select
to authenticated
using (
  bucket_id = 'videos-conteos'
  and (storage.foldername(name))[1] in (
    select id::text from datos_campo where productor_id = auth.uid()
  )
);

alter table videos_conteo enable row level security;
alter table video_marcas enable row level security;

create policy "usuarios ven videos_conteo de su campo"
on videos_conteo for select to authenticated
using (
  conteo_id in (
    select c.id from conteos c
    join datos_campo dc on dc.id = c.campo_id
    where dc.productor_id = auth.uid()
  )
);

create policy "usuarios ven video_marcas de su campo"
on video_marcas for select to authenticated
using (
  conteo_id in (
    select c.id from conteos c
    join datos_campo dc on dc.id = c.campo_id
    where dc.productor_id = auth.uid()
  )
);
```

5. Lo que vimos probando en la netbook — pausa antes de la demo

En los videos que mandaste se ve el script funcionando bien hasta el momento de subir el video: `requests.post(...)` hacia Supabase Storage se queda colgado (traceback dentro de `ssl.py`, en pleno envío) y hay que interrumpirlo con Ctrl+C — pasó dos veces seguidas. El video se graba bien en la netbook, el problema es específicamente al subirlo, probablemente por el wifi de la escuela. Antes de la presentación, conviene:

- Probar la subida con otra red (celular como punto de acceso, por ejemplo) para confirmar si el problema es de esa red en particular.
- Si vas a dejarlo corriendo el día de la demo, considerar guardar el video localmente igual (ya lo hace) para no perder nada si la subida falla — la fila en `videos_conteo` se puede crear después a mano si hace falta.
- Opcional, si da tiempo: envolver el POST de subida en un reintento simple (2-3 intentos con una pausa entre medio) en vez de una sola llamada con `timeout=120`.

6. Qué arquitectura estás siguiendo (y qué falta decidir)

El proyecto tiene un documento con el análisis completo de cómo grabar los conteos (`Funcion_Camara-CowControl.pdf`, en la misma carpeta que este PDF): se evaluaron dos caminos y **todavía no se decidió cuál usar** — a propósito, Santiago prefirió dejarlo abierto:

- **Camino A** — clips cortos de 2-3 segundos por lectura, sin depender de subir un video largo.
- **Camino B** — un solo video completo con el nombre/UID superpuesto en el momento justo, y la app salta directo a ese segundo (como un timestamp de YouTube). **Esto es lo que ya estás armando vos.**

El punto más importante de ese documento para tu enfoque: Supabase Storage tiene un límite de **50MB por archivo** (y 1GB en total en el plan actual). Un conteo largo grabado en video completo podría acercarse a ese límite más rápido que una tanda de clips cortos — no es un problema hoy, pero es la razón por la que conviene confirmar con Santiago que el Camino B es el elegido antes de dejarlo como definitivo.

7. Checklist para probar de punta a punta

- Aplicar los cambios de nombres de tabla/columna de la sección 2 en grabador_conteo.py.
- Confirmar con Santiago el tema de la columna "nombre" (2.5) antes de pedir esa migración.
- Crear las tablas videos_conteo/video_marcas (sección 3) y el bucket privado solo después de coordinarlo con Santiago.
- Aplicar las políticas de RLS corregidas (sección 4).
- Probar la subida con una red distinta a la de la escuela antes de confiar en la demo con esa conexión.
- Iniciar un conteo de prueba desde la app, pasar un tag, pausar, reanudar, y finalizar — confirmar que el video queda sincronizado y sin el tramo pausado (esto ya lo tenías bien diseñado, solo hace falta confirmarlo end-to-end con los nombres corregidos).